

Eugene Moshchyn

Dr. Qi Cheng

CS 4413, Section 1

April 14, 2026

Assignment 8

Exercise 20.1-2 Give an adjacency-list representation for a complete binary tree on 7 vertices. Give an equivalent adjacency-matrix representation. Assume that the edges are undirected and that the vertices are numbered from 1 to 7 as in a binary heap.

Solution

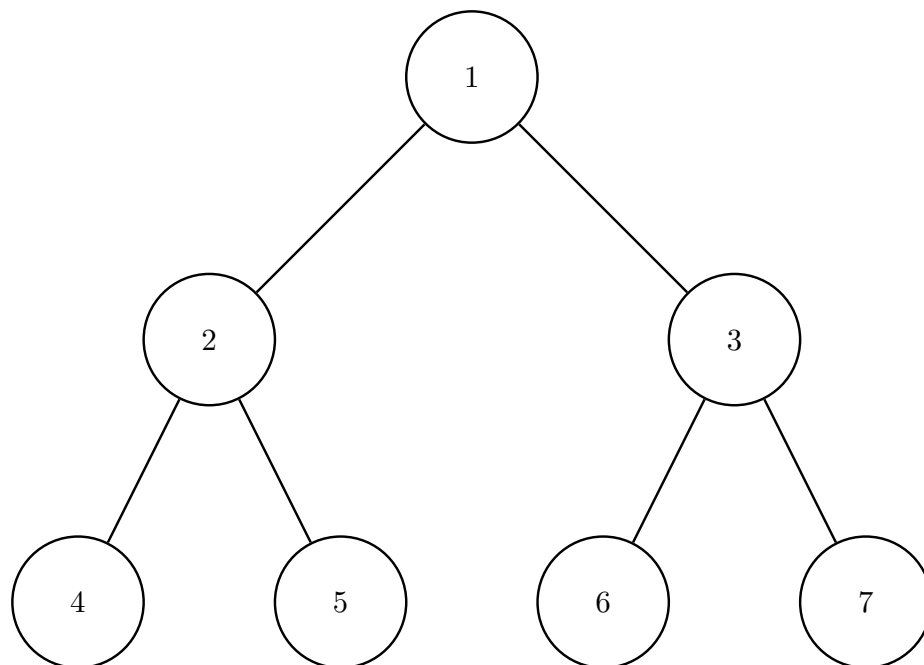


Figure 1: Complete binary tree consisting of seven vertices

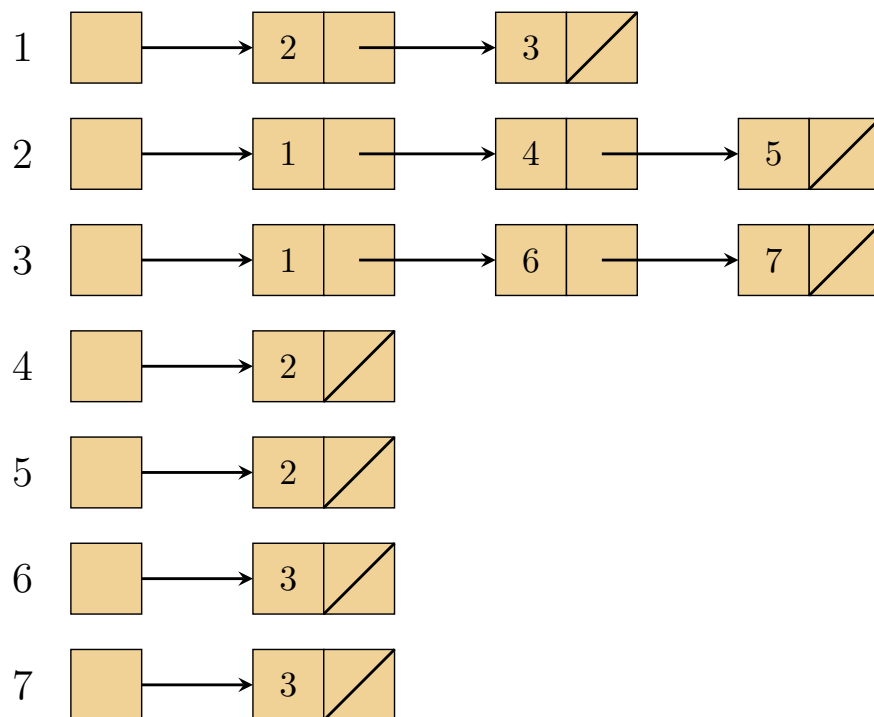


Figure 2: An adjacency-list representation of the complete binary tree shown in Figure 1

	1	2	3	4	5	6	7
1	0	1	1	0	0	0	0
2	1	0	0	1	1	0	0
3	1	0	0	0	0	1	1
4	0	1	0	0	0	0	0
5	0	1	0	0	0	0	0
6	0	0	1	0	0	0	0
7	0	0	1	0	0	0	0

Figure 3: The adjacency-matrix representation of the complete binary tree shown in Figure 1

Exercise 20.2-7 There are two types of professional wrestlers: “faces” (short for “babyfaces,” i.e., “good guys”) and “heels” (“bad guys”). Between any pair of professional wrestlers, there may or may not be a rivalry. You are given the names of n professional wrestlers and a list of r pairs of wrestlers for which there are rivalries. Give an $O(n + r)$ -time algorithm that determines whether it is possible to designate some of the wrestlers as faces and the remainder as heels such that each rivalry is between a face and a heel. If it is possible to perform such a designation, your algorithm should produce it.

Solution

- Create a graph $G = (V, E)$ where each wrestler is represented by a vertex and each rivalry is represented by an undirected edge.
- Use BREADTH-FIRST SEARCH to traverse the graph. If the graph is disconnected, repeat the process for each unvisited node to ensure all components are processed.
- During the BFS, assign a distance to each vertex. Designate wrestlers at even distances as **faces** and wrestlers at odd distances as **heels**.
- Finally, traverse all edges $(u, v) \in E$ to verify that no two wrestlers of the same type (e.g., face-face or heel-heel) share an edge. If such an edge exists, a valid designation is impossible.

Exercise 20.3-9 Modify the pseudocode for depth-first search so that it prints out every edge in the directed graph G , together with its type. Show what modifications, if any, you need to make if G is undirected.

```

DFS-VISIT( $G, u$ )
1   $time = time + 1$                                 // white vertex  $u$  has just
                                                    // been discovered
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each vertex  $v$  in  $G.Adj[u]$                     // explore each edge  $(u, v)$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $time = time + 1$ 
9   $u.f = time$ 
10  $u.color = \text{BLACK}$                                 // blacken  $u$ ; it is finished

```

Figure 4: Original DFS-VISIT implementation from Section 20.3 (Depth-first search), p. 565.

Solution

```

DFS-VISIT( $G, u$ )
1   $time = time + 1$                                 // white vertex  $u$  has just
                                                    // been discovered
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each vertex  $v$  in  $G.Adj[u]$                 // explore each edge  $(u, v)$ 
5      if  $v.color == \text{WHITE}$ 
6          print “( $u, v$ ): tree edge”
7           $v.\pi = u$ 
8          DFS-VISIT( $G, v$ )
9      elseif  $v.color == \text{GRAY}$ 
10         print “( $u, v$ ): back edge”
11     elseif  $u.d < v.d$ 
12         print “( $u, v$ ): forward edge”
13     else
14         print “( $u, v$ ): cross edge”
15   $time = time + 1$ 
16   $u.f = time$ 
17   $u.color = \text{BLACK}$                             // blacken  $u$ ; it is finished

```

Figure 5: Modified DFS-VISIT algorithm, with changes highlighted in red.